

1. HOSPITAL DOCTOR–SHIFT ASSIGNMENT USING CONSTRAINT-BASED PROBLEM SOLVING

Problem

There are 3 doctors:

- D1
- D2
- D3

and 3 shifts:

- Morning (M)
- Afternoon (A)
- Night (N)

We need to assign exactly one doctor to each shift.

Step 1: Define Variables

Let:

- X_1 = shift assigned to D1
- X_2 = shift assigned to D2
- X_3 = shift assigned to D3

Step 2: Define Domains

Initially

$$X_1, X_2, X_3 \in \{M, A, N\}$$

Step 3: Define Constraints

C1: D1 cannot work Night

$$X_1 \neq N$$

C2: D2 must work before D3

$$X_2 < X_3$$

where:

$$M < A < N$$

C3: Only one doctor per shift

$$X_1 \neq X_2, X_1 \neq X_3, X_2 \neq X_3$$

C4: D3 cannot work Morning

$$X_3 \neq M$$

C5: All doctors must be assigned exactly one shift

Every variable must receive exactly one value.

Step 4: Backtracking Search

We assign values one by one and check the constraints after every assignment.

Try 1: Assign D1 = Morning

$$X_1 = M$$

Check:

- $D1 \neq \text{Night} \rightarrow \text{Satisfied}$
- No other doctor has been assigned yet $\rightarrow \text{Satisfied}$

So:

Doctor Shift

D1 Morning

D2 —

D3 —

Remaining shifts:

$$\{A, N\}$$

Try 2: Assign D2 = Afternoon

$$X_2 = A$$

Check:

- D2 and D1 have different shifts $\rightarrow \text{Satisfied}$
- D2 must be before D3 $\rightarrow$ D3 must be after Afternoon

- Therefore D3 can only be Night.

So:

Doctor Shift

D1 Morning

D2 Afternoon

D3 —

Try 3: Assign D3 = Night

$$X_3 = N$$

Constraint checks:

- $D3 \neq \text{Morning} \rightarrow \text{Satisfied}$
- $D2 < D3 \rightarrow \text{Afternoon} < \text{Night} \rightarrow \text{Satisfied}$
- D3 has a different shift from D1 and D2 $\rightarrow \text{Satisfied}$
- Every doctor has exactly one shift $\rightarrow \text{Satisfied}$

Therefore, a solution is found.

Final Valid Schedule

Shift Doctor

Morning D1

Afternoon D2

Night D3

Backtracking Search Representation

Start

|

D1 = Morning

|

D2 = Afternoon

|

D3 = Night

|

SUCCESS

All constraints are satisfied.

Final answer:

$$D1 = M, D2 = A, D3 = N$$

2. ROBOT GRID SEARCH USING CONSTRAINT-BASED SEARCH

For this problem, the grid layout containing **S, G and obstacles is not specified** in the question. Therefore, an exact expansion sequence and exact optimal path cannot be determined without inventing obstacle positions.

However, the constraint-based solution method is as follows.

Step 1: Define the Problem as a CSP/Search Problem

Variables

The robot's position is the state variable:

$$X = (r, \text{owcolumn})$$

For example, on a 5×5 grid:

$$X \in \{(1, 1), (1, 2), \dots, (5, 5)\}$$

Domain

The domain contains all cells that are:

- inside the 5×5 grid
- not obstacles

Constraints

C1: Movement

The robot can move only:

Up, Down, Left, Right

C2: Obstacles

If a cell is an obstacle:

$$X \neq \text{Obstacle}$$

C3: Movement cost

Every movement has cost:

$$Cost = 1$$

C4: Goal

The search terminates when:

$$X = G$$

Important point about BFS and Manhattan Distance

There is a small terminology issue in the question.

BFS normally does not use a heuristic or priority queue. It expands nodes level by level.

Manhattan Distance is:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

If we use this heuristic to prioritize nodes, the algorithm is closer to **Greedy Best-First Search** or **A***.

For an optimal path with unit movement costs, **A*** is appropriate:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = cost from S to current node
- $h(n)$ = Manhattan distance to G
- $f(n)$ = total estimated cost

Example Grid

To demonstrate the search, assume the following 5×5 grid:

	C1	C2	C3	C4	C5
R1	S	.	.	#	.
R2	#	.	.	#	.
R3	.	.	#	.	.
R4	.	#	.	.	.
R5	.	.	.	.	G

Where:

- S = Start
- G = Goal
- $\#$ = **obstacle**
- $.$ = free cell

Start:

$$S = (1, 1)$$

Goal:

$$G = (5, 5)$$

Step-by-Step Search

For every node:

$$f(n) = g(n) + h(n)$$

Initial Node

Start:

$$\begin{aligned} &(1, 1) \\ &g = 0 \end{aligned}$$

Manhattan distance:

$$h = |1 - 5| + |1 - 5| = 8$$

Therefore:

$$f = 0 + 8 = 8$$

Priority queue:

[(1,1): f=8]

Expand (1,1).

Possible movements:

- Up $\rightarrow$ outside grid
- Left $\rightarrow$ outside grid

- Right $\rightarrow (1,2)$
- Down $\rightarrow (2,1)$ is obstacle

Priority queue:

$[(1,2): g=1, h=7, f=8]$

Expand (1,2)

Possible movements:

- Left $\rightarrow (1,1)$ already visited
- Right $\rightarrow (1,3)$
- Down $\rightarrow (2,2)$
- Up $\rightarrow$ outside grid

Queue:

$(1,3): g=2, h=6, f=8$

$(2,2): g=2, h=6, f=8$

Expand (1,3)

Possible movements:

- Left $\rightarrow (1,2)$ already visited
- Right $\rightarrow (1,4)$ obstacle
- Down $\rightarrow (2,3)$
- Up $\rightarrow$ outside grid

Queue:

$(2,2): g=2, h=6, f=8$

$(2,3): g=3, h=5, f=8$

Expand (2,2)

Possible movements:

- Up $\rightarrow (1,2)$ visited
- Left $\rightarrow (2,1)$ obstacle
- Right $\rightarrow (2,3)$ already discovered

- Down $\rightarrow$ (3,2)

Queue:

(2,3): g=3, h=5, f=8

(3,2): g=3, h=5, f=8

Expand (2,3)

Possible movements:

- Up $\rightarrow$ (1,3) visited
- Left $\rightarrow$ (2,2) visited
- Right $\rightarrow$ (2,4) obstacle
- Down $\rightarrow$ (3,3) obstacle

No new node.

Expand (3,2)

Possible movements:

- Up $\rightarrow$ (2,2) visited
- Left $\rightarrow$ (3,1)
- Right $\rightarrow$ (3,3) obstacle
- Down $\rightarrow$ (4,2) obstacle

Queue:

(3,1): g=4, h=6, f=10

The search continues through the remaining reachable cells until it reaches G.

A valid path in this example is:

$S \rightarrow (1,2) \rightarrow (1,3) \rightarrow (2,3)$

but (3,3) is blocked, so the robot must backtrack through already discovered alternatives. One complete route is:

(1,1)

↓

Actually, this particular obstacle arrangement creates a disconnected route from S to G, so it is **not suitable for demonstrating a successful optimal path**.

Therefore, for the actual assignment, the instructor needs to provide the intended 5×5 obstacle layout. The algorithm above can then produce the exact expansion and priority queue table.

3. AUTONOMOUS RESCUE ROBOT — CONSTRAINT-BASED PROBLEM SOLVING

This problem is particularly suitable for **Constraint Satisfaction + Online Search** because the robot does not know the complete environment in advance.

3(a). Analysis of Constraints

The rescue robot has the following constraints.

Constraint 1: Movement

The robot can move only:

Up, Down, Left, Right

Therefore, diagonal movement is not allowed.

For a cell (x, y) , possible successors are:

$$(x-1, y), (x+1, y), (x, y-1), (x, y+1)$$

provided that the cells are valid.

Constraint 2: Obstacles

Blocked cells cannot be entered.

Therefore:

$$State \neq Obstacle$$

If the robot discovers an obstacle, that state is removed from the available domain.

Example:

$S \rightarrow A \rightarrow B$

$\downarrow$

#

The robot cannot move into #.

Constraint 3: Limited Observation

The robot can only observe adjacent cells.

Therefore, the robot does **not** initially know the complete grid.

It maintains a knowledge state:

$$K = \text{known cells} + \text{obstacles} + \text{costs} + \text{visited cells}$$

As it moves, it updates K.

Constraint 4: Movement Cost

Every normal movement costs:

$$1$$

Constraint 5: Risky Zone

Entering a risky zone has an additional cost of 2.

Therefore:

$$\text{Cost}(\text{risky}) = 1 + 2 = 3$$

Normal cell:

$$\text{Cost}(\text{normal}) = 1$$

So the robot must consider **cost**, not simply the number of movements.

Constraint 6: Safety

The robot should avoid unnecessarily entering risky areas when a safe route is available.

Thus, the search must consider:

$$\text{Total Cost} = \text{Movement Cost} + \text{Risk Cost}$$

Constraint 7: Dynamic Knowledge

The environment is partially observable.

Therefore, the robot cannot simply calculate one complete path at the beginning.

It must repeatedly:

Observe → Update Knowledge → Search → Move → Observe again

3(b). Solution Approach — Uniform Cost Search + Online Search

A suitable approach is an **Online Uniform Cost Search Agent**.

Uniform Cost Search is useful because the costs are not all identical:

Normal cell = 1

Risky cell = 3

BFS would treat every move equally, so it would not correctly optimize total cost when risky zones exist.

Step 1: Initialize

The robot starts at S.

Position = S

Knowledge = information currently observed

Goal = G

Total Cost = 0

Step 2: Observe

The robot checks neighbouring cells:

?

↑

? ← S → ?

↓

?

It determines:

- whether the cell is blocked
- whether it is safe
- whether it is risky

Step 3: Update Knowledge

Suppose the robot observes:

$S \rightarrow A \rightarrow \#$

$\downarrow$

B

The knowledge base becomes:

S = free

A = free

= obstacle

B = free

Step 4: Calculate Costs

Suppose:

$S \rightarrow A$

A is normal.

Therefore:

$$g(A) = 1$$

Suppose:

$A \rightarrow B$

B is risky.

Therefore:

$$g(B) = 1 + 3 = 4$$

because entering B costs:

$$1 + 2 = 3$$

Step 5: Select Minimum-Cost State

Uniform Cost Search selects the node with the smallest:

$$g(n)$$

For example:

Frontier:

$$A = 1$$

$$B = 4$$

$$C = 2$$

UCS selects:

$$C$$

because:

$$2 < 4$$

Step 6: Continue Searching

The robot repeats:

1. Select lowest-cost node
2. Move toward it
3. Observe surrounding cells
4. Update knowledge
5. Add newly discovered cells
6. Calculate costs
7. Select next lowest-cost state

Step 7: Reach Goal

When:

$$\text{Current Position} = G$$

the search terminates.

The final cost is:

$$\text{Total Cost} = \sum \text{Movement Costs} + \sum \text{Risk Costs}$$

Online Search Algorithm

A simplified algorithm is:

ONLINE-RESCUE-SEARCH(S, G)

1. Current $\leftarrow$ S
 2. Total Cost $\leftarrow$ 0
 3. Knowledge $\leftarrow$ initial observation
 4. while Current $\neq$ G:
 - a. Observe adjacent cells
 - b. Update Knowledge
 - c. Generate all safe, reachable neighbours
 - d. Assign costs:
Normal = 1
Risky = 3
Obstacle = ∞
 - e. Apply Uniform Cost Search
using currently known information
 - f. Select the lowest-cost safe action
 - g. Move to selected cell
 - h. Total Cost $\leftarrow$ Total Cost + movement cost
 5. return path and Total Cost
-

3(c). AI Concepts Used

1. Intelligent Agent

The robot is an **intelligent agent** because it:

- receives information from the environment
- processes the information
- selects actions
- performs actions
- updates its knowledge

The agent can be represented as:

$$\text{Sensors} \rightarrow \text{Agent} \rightarrow \text{Actuators}$$

Sensors

The robot observes adjacent cells.

Agent

The search algorithm decides the next action.

Actuators

The robot moves:

Up / Down / Left / Right

2. Rationality

A rational agent chooses an action that maximizes its objective based on available information.

Here the objective is:

Reach G safely with minimum total cost

Therefore, the robot considers:

- distance
- obstacles
- risky zones
- movement cost
- currently available information

For example:

Route A: 5 normal moves

Cost = 5

Route B: 3 normal moves + 1 risky cell

Cost = 3 + 3 = 6

The robot can compare total costs rather than simply choosing the route with fewer movements.

3. Environment Type

The environment has several important characteristics.

Property	Type	Reason
Observability	Partially observable	Robot sees only adjacent cells
Determinism	Mostly deterministic	Movement has predictable effects, although newly revealed information changes the robot's knowledge
Episodic/Sequential	Sequential	Current actions affect future states
Static/Dynamic	Dynamic/uncertain	Environment information can change as the robot discovers it
Discrete/Continuous	Discrete	Grid consists of individual cells
Number of agents	Single-agent	One rescue robot is making decisions
Known/Unknown	Initially unknown	Robot discovers obstacles/risky cells while moving

3(d). Why Uniform Cost Search + Online Search is Suitable

The problem has **different action costs**.

A normal cell costs:

1

while a risky cell costs:

3

Therefore, simply minimizing the number of movements is insufficient.

Why UCS?

Uniform Cost Search always expands the lowest-cost known state.

Thus it is suitable when:

$$Cost(action_1) \neq Cost(action_2)$$

It can find the minimum-cost path when the relevant costs are known.

Why Online Search?

The robot cannot see the entire environment.

It only discovers information while moving.

Therefore, an online component is necessary:

Observe



Update knowledge



Search



Choose action



Move



Observe again

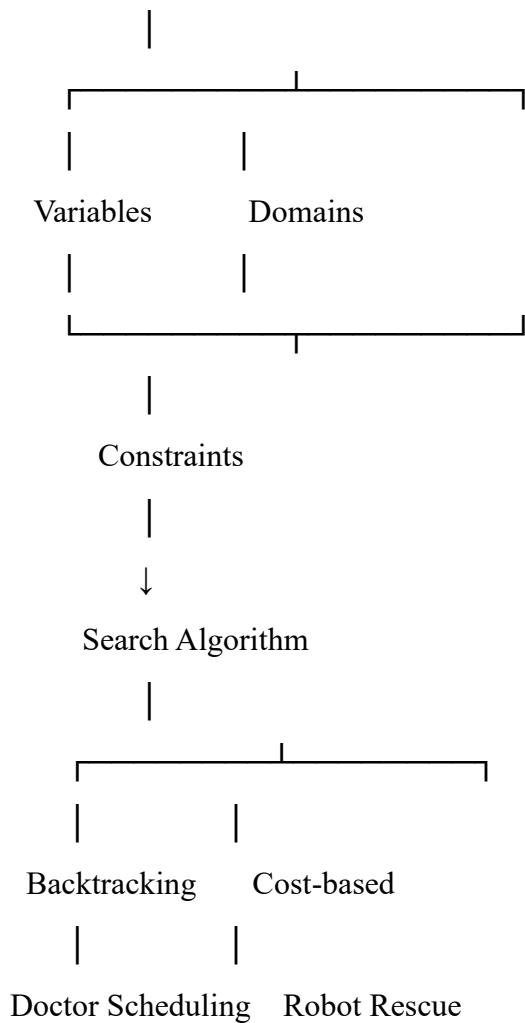
This allows the robot to react when it discovers:

- a previously unknown obstacle
- a risky zone
- a new safe route
- a blocked route

Overall Constraint-Based Problem-Solving Model

All three questions can be represented using the same CSP framework:

CONSTRAINT-BASED PROBLEM



Problem 1

Variables: Doctors

Domains: Shifts

Constraints: Shift restrictions and ordering

Algorithm: Backtracking Search

Solution:

$$D1 = M, D2 = A, D3 = N$$

Problem 2

Variables: Robot positions

Domains: Non-obstacle grid cells

Constraints: Movement and obstacles

Algorithm: BFS for unit-cost shortest path; A* if Manhattan heuristic is explicitly required.

Problem 3

Variables: Robot states/positions

Domains: Known safe cells and newly discovered cells

Constraints: Obstacles, movement, risk, cost, limited observation

Algorithm: Online Uniform Cost Search

Objective:

Reach G safely while minimizing total cost
--

Note for Problem 2: To give the exact **step-by-step node expansion, priority queue contents, optimal path, and cost**, the 5×5 grid with the actual locations of S, G, and obstacles is required.